Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 13.11.2023.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2024_Test1_GrupaF_IT1g2023**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Quick Sort*

Napisati ***quick*** metodu ***Quick Sort*** algoritma za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomNumberGen*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***QuickSort*** napisati implementaciju za metodu ***private static void quickSort(int[] arr, int lower, int upper)***
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – Brisanje neparnih brojeva iz steka, proizvod preostalih

Implementirati metodu koja prima niz celih brojeva. Metoda treba da koristi strukturu podataka steka (klasa **Stack**) za organizaciju i manipulaciju elementima, i treba da izvrši sledeće korake:

1. **Uklanjanje neparnih brojeva**: Prolazom kroz stek, ukloniti sve brojeve koji neparni
2. **Računanje proizvoda preostalih brojeva**: Izračunati proizvod svih brojeva koji su ostali u steku nakon uklanjanja neparnih brojeva.
3. **Vraćanje rezultata**: Kao rezultat, metoda treba da vrati izračunatu proizvod preostalih brojeva u steku.

Dodatni zahtevi za rešavanje zadatka:

- Zadatak rešavati pomoću strukture ***steka*** (klasa ***Stack***) koja je data u okviru šablona zadatka.
- Zadatak rešavati u okviru metode ***public int RemoveOddAndReturnProduct (int[] array)***
- Nakon što se uklone neparni brojevi iz steka i izračuna proizvod preostalih brojeva, stek treba da ostane u istom stanju kao pre početnog procesa. To znači da preostali brojevi moraju biti vraćeni u stek u originalnom redosledu, a svi izmenjeni elementi (oni koji su uklonjeni) ne smeju se vratiti.
- Algoritam je neophodno napisati sa linearnom vremenskom kompleksnošću O(n).

Primer:

- Input: [10, 15, 7, 30, 2, 8]

- Output: Proizvod preostalih brojeva: 4800
  Preostali brojevi u steku: 10, 30, 2, 8

- Input: [5, 2, 8, 3, 4, 6]

- Output: Proizvod preostalih brojeva: 384
  Preostali brojevi u steku: 2, 8, 4, 6

Zadatak 3 – *Razlika između maksimalne vrednosti lokalnih maksimuma i minimalne vrednosti lokalnih minimuma*

Implementirati metodu koja prima povezanu listu celih brojeva i pronalazi sve lokalne maksimume i lokalne minimume, zatim izračunava razliku između maksimalne vrednosti lokalnih maksimuma i minimalne vrednosti lokalnih minimuma.

Za definiciju:

- **Lokalni maksimum** je broj koji je veći od prethodnog i sledećeg broja.
- **Lokalni minimum** je broj koji je manji od prethodnog i sledećeg broja.

Ako nema lokalnih maksimuma ili minimuma, metoda treba da vrati -1.

<u>Dodatni zahtevi za rešavanje zadatka:</u>

- Zadatak rešavati pomoću strukture liste (klasa ***LinkedList***) koja je data u okviru šablona zadatka.
- Ne koristiti dodatne kolekcije poput liste ili niza, koristiti samo povezanu listu.
- Zahtevan potpis metode: ***public int findLocalMaxMinDifference()***

<u>Primer1:</u>

Input: [1, 3, 2, 5, 6, 4, 7, 3]

- Lokalni maksimumi: 3, 6, 7 (indeksi 1, 4, 6)
- Lokalni minimumi: 2, 4 (indeksi 2, 5)
- Maksimalna vrednost lokalnih maksimuma je 7, a minimalna vrednost lokalnih minimuma je 2.
- Razlika između 7 i 2 je 5.

Output: 5

<u>Primer2:</u>

Input: [1, 2, 3, 4, 5]

- Nema lokalnih minimuma i maksimuma

Output: -1